



LABORATORIO: FILE PATH TRAVERSAL - TRAVERSAL SEQUENCES STRIPPED WITH SUPERFLUOUS URL- DECODE

CNO IV - SEGURIDAD INFORMÁTICA

MELANIE ALONDRA ALBA DIEZ

185583

ING. EN TECNOLOGÍAS DE LA INFORMACIÓN



Índice

Especificaciones del laboratorio.....	2
Marco teórico.....	2
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento	4
1. Acceso al laboratorio mediante Burp Suite	4
2. Identificación de las peticiones de imágenes.....	4
3. Envío de la petición a Repeater	5
4. Modificación del parámetro vulnerable	6
5. Análisis de la respuesta	6
6. Confirmación de resolución	7
Explicación del payload.....	8
Mitigación recomendada	8
Referencias	9

Especificaciones del laboratorio

Nombre del laboratorio: File path traversal, traversal sequences stripped with superfluous URL-decode.

Nivel: PRACTITIONER

Tipo de explotación: File Path Traversal (Directory Traversal) mediante bypass utilizando doble codificación URL.

Marco teórico

El **File Path Traversal**, también conocido como **Directory Traversal**, es una vulnerabilidad que ocurre cuando una aplicación utiliza datos proporcionados por el usuario para construir rutas de archivos sin comprobar adecuadamente que el recurso solicitado permanezca dentro del directorio autorizado. Como consecuencia, un atacante podría manipular la ruta utilizada por la aplicación para intentar acceder a archivos ubicados en otras partes del sistema.

Una forma convencional de explotar esta vulnerabilidad consiste en utilizar la secuencia **../**, donde **..** hace referencia al directorio superior y **/** funciona como separador. Al repetir esta secuencia varias veces es posible retroceder dentro de la estructura de directorios hasta alcanzar otros archivos del sistema.

Para prevenir este comportamiento, algunas aplicaciones eliminan o bloquean las secuencias de recorrido antes de utilizar el valor proporcionado por el usuario. Sin embargo, esta protección puede resultar insuficiente cuando existen diferentes etapas de procesamiento de la entrada, especialmente cuando se realizan operaciones adicionales de **URL decoding**.

En una URL, determinados caracteres pueden representarse mediante **percent-encoding**. Por ejemplo, el carácter **/** puede representarse como **%2f**. A su vez, el símbolo **%** puede codificarse como **%25**, por lo que **%2f** puede representarse mediante una segunda codificación como **%252f**.

Este comportamiento da origen a la **doble codificación URL (Double URL Encoding)**. Una entrada como: **../%252f**, puede transformarse después de una primera decodificación en: **../%2f**, y, después de una decodificación adicional, producir finalmente: **../**.

Si una aplicación comprueba la entrada antes de que ocurra una etapa adicional de decodificación, es posible que el filtro no reconozca inicialmente la secuencia peligrosa. Posteriormente, una operación adicional de URL decoding puede reconstruir la secuencia de recorrido de directorios que será interpretada por la aplicación.

Desde el modelo de seguridad **CIA (Confidentiality, Integrity and Availability)**, esta vulnerabilidad afecta principalmente a la **confidencialidad**, debido a que puede permitir el acceso a información que debería permanecer fuera del alcance del usuario, como archivos del sistema, configuraciones, código fuente, registros o credenciales.

La prevención requiere que la aplicación no dependa únicamente de filtros aplicados sobre la representación inicial de la entrada. Los datos deben ser normalizados y

validados de manera consistente, comprobando además que la ruta final permanezca dentro del directorio autorizado.

Objetivo del laboratorio

Identificar y explotar de manera controlada una vulnerabilidad de **File Path Traversal** presente en el mecanismo utilizado por una aplicación web para cargar imágenes de productos.

Mediante **Burp Suite**, se busca analizar y modificar una petición HTTP que contiene el parámetro **filename**. La aplicación bloquea las secuencias convencionales de recorrido de directorios; sin embargo, posteriormente realiza una operación adicional de URL decoding, permitiendo utilizar una entrada doblemente codificada para evadir el mecanismo de protección.

El laboratorio se considera resuelto al modificar el parámetro mediante el payload: **..%252f..%252f..%252fetc/passwd**, y conseguir que el servidor devuelva el contenido del archivo **/etc/passwd**, demostrando que el mecanismo de validación puede ser evadido mediante doble codificación URL.

Explicación técnica de la vulnerabilidad

La aplicación carga las imágenes de los productos mediante peticiones HTTP dirigidas al endpoint **/image**, utilizando el parámetro **filename** para indicar qué archivo debe recuperar el servidor.

Una petición legítima presenta una estructura similar a: **GET /image?filename=73.jpg**

El valor de **filename** es utilizado por la aplicación para localizar el recurso solicitado. En este laboratorio existe un mecanismo de protección que impide utilizar directamente las secuencias convencionales de recorrido de directorios.

Por esta razón, utilizar directamente: **../../../../etc/passwd**, no resulta suficiente para evadir el filtro.

La vulnerabilidad aparece debido al orden en que la aplicación procesa la entrada. El mecanismo de protección comprueba o elimina las secuencias de traversal, pero posteriormente la aplicación realiza una operación adicional de **URL decoding**.

Para aprovechar este comportamiento se puede representar el separador **/** mediante doble codificación.

De esta manera, el payload utilizado es: **..%252f..%252f..%252fetc/passwd**.

La representación **%252f** no aparece inicialmente como un **/** convencional. Después de las operaciones de decodificación realizadas durante el procesamiento de la solicitud, puede convertirse nuevamente en **%2f** y posteriormente en **/**, reconstruyendo las secuencias **../**.

El resultado efectivo permite obtener una ruta equivalente a: **../../../../etc/passwd**.

De esta forma, es posible abandonar el directorio destinado a las imágenes y alcanzar el archivo **/etc/passwd**.

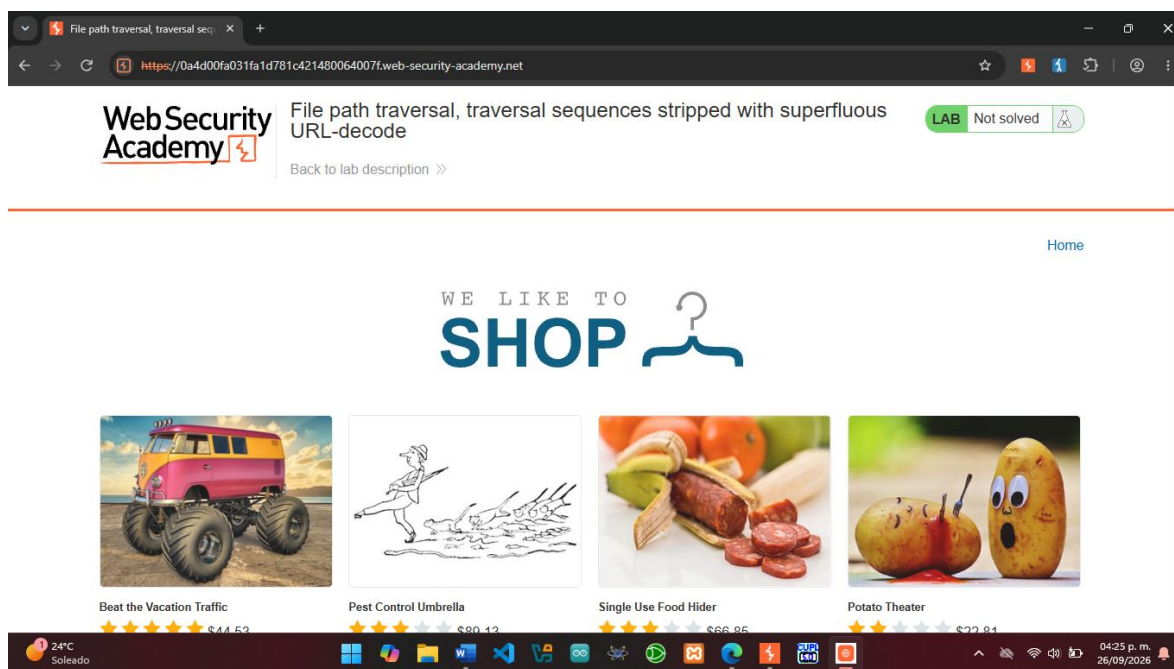
Por lo tanto, la vulnerabilidad se produce porque la aplicación valida la entrada en una etapa diferente de aquella en la que finalmente es interpretada, permitiendo que una representación codificada evada el filtro y posteriormente se transforme en una secuencia de traversal válida.

Procedimiento

1. Acceso al laboratorio mediante Burp Suite

Se abrió **Burp Suite Community Edition** y, desde la pestaña **Proxy**, se inició el navegador Chromium integrado mediante la opción **Open Browser**. Este navegador se encuentra configurado para dirigir su tráfico a través del proxy de Burp Suite, permitiendo observar las peticiones y respuestas HTTP generadas durante la navegación.

Posteriormente, desde dicho navegador se accedió a la instancia asignada del laboratorio **File path traversal, traversal sequences stripped with superfluous URL-decode** de PortSwigger Web Security Academy.



2. Identificación de las peticiones de imágenes

Desde **Proxy > HTTP history** se analizaron las solicitudes generadas por la aplicación. Se observó que la carga de las imágenes de los productos producía múltiples peticiones **GET** dirigidas al endpoint **/image**, cada una de ellas utilizando el parámetro **filename** para indicar el archivo que debía ser recuperado. Por ejemplo: **GET /image?filename=73.jpg**.

Este comportamiento permitió identificar **filename** como un parámetro susceptible de prueba, debido a que su valor determina el archivo solicitado al servidor.

#	Host	Met...	URL	Params	Edited	Status code	Length	MIME type
1	https://0a4d00fa031fa1d78...	GET	/			200	10614	HTML
6	https://0a4d00fa031fa1d78...	GET	/image?filename=73.jpg	✓		200	253369	JPEG
7	https://0a4d00fa031fa1d78...	GET	/image?filename=42.jpg	✓		200	98514	JPEG
10	https://0a4d00fa031fa1d78...	GET	/image?filename=20.jpg	✓		200	184511	JPEG
11	https://0a4d00fa031fa1d78...	GET	/image?filename=19.jpg	✓		200	195225	JPEG
13	https://0a4d00fa031fa1d78...	GET	/image?filename=31.jpg	✓		200	127667	JPEG
16	https://0a4d00fa031fa1d78...	GET	/academyLabHeader			101	147	
17	https://0a4d00fa031fa1d78...	GET	/image?filename=19.jpg	✓		200	195225	JPEG
19	https://0a4d00fa031fa1d78...	GET	/image?filename=20.jpg	✓		200	184511	JPEG
20	https://0a4d00fa031fa1d78...	GET	/image?filename=42.jpg	✓		200	98514	JPEG
21	https://0a4d00fa031fa1d78...	GET	/image?filename=73.jpg	✓		200	253369	JPEG

3. Envío de la petición a Repeater

Se seleccionó una de las peticiones dirigidas a **/image** y se utilizó la opción **Send to Repeater**, obtenida del menú desplegado al dar clic derecho sobre la petición.

De esta manera, Burp Suite copió la petición al módulo **Repeater**, permitiendo modificar manualmente sus parámetros y reenviarla al servidor para analizar las respuestas obtenidas.

#	Host	Met...	URL	Params	Edited	Status code	Length	MIME type	Extension
1	https://0a4d00fa031fa1d78...	GET	/			200	10614	HTML	
6	https://0a4d00fa031fa1d78...	GET	/image?filename=73.jpg			200	253369	JPEG	
7	https://0a4d00fa031fa1d78...	GET	/image?filename=42.jpg			200	98514	JPEG	
10	https://0a4d00fa031fa1d78...	GET	/image?filename=20.jpg			200	184511	JPEG	
11	https://0a4d00fa031fa1d78...	GET	/image?filename=19.jpg			200	195225	JPEG	
13	https://0a4d00fa031fa1d78...	GET	/image?filename=31.jpg			200	127667	JPEG	
16	https://0a4d00fa031fa1d78...	GET	/academyLabHeader			101	147		
17	https://0a4d00fa031fa1d78...	GET	/image?filename=19.jpg			200	195225	JPEG	
19	https://0a4d00fa031fa1d78...	GET	/image?filename=20.jpg			200	184511	JPEG	
20	https://0a4d00fa031fa1d78...	GET	/image?filename=42.jpg			200	98514	JPEG	
21	https://0a4d00fa031fa1d78...	GET	/image?filename=73.jpg			200	253369	JPEG	

Request

Pretty Raw Hex

```

1 GET /image?filename=73.jpg HTTP/2
2 Host: 0a4d00fa031fa1d781c421480064007f.web-security
3 Cookie: session=ZFbiVgRprUiTwp76fK7ShwYx2f750bxw
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-ES,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors

```

https://0a4d00fa031fa1d781c421480064007f.web-security.net/image?filename=73.jpg

Add to scope

Scan

Send to Intruder Ctrl+I

Send to Repeater Ctrl+R

Send to Sequencer

Send to Organizer Ctrl+O

Send to Comparer (request)

Send to Comparer (response)

Open response in browser

Request in browser >

Record an issue [Pro version only] >

Engagement tools [Pro version only] >

Show new history window

Add notes

Highlight > to

Delete item iA

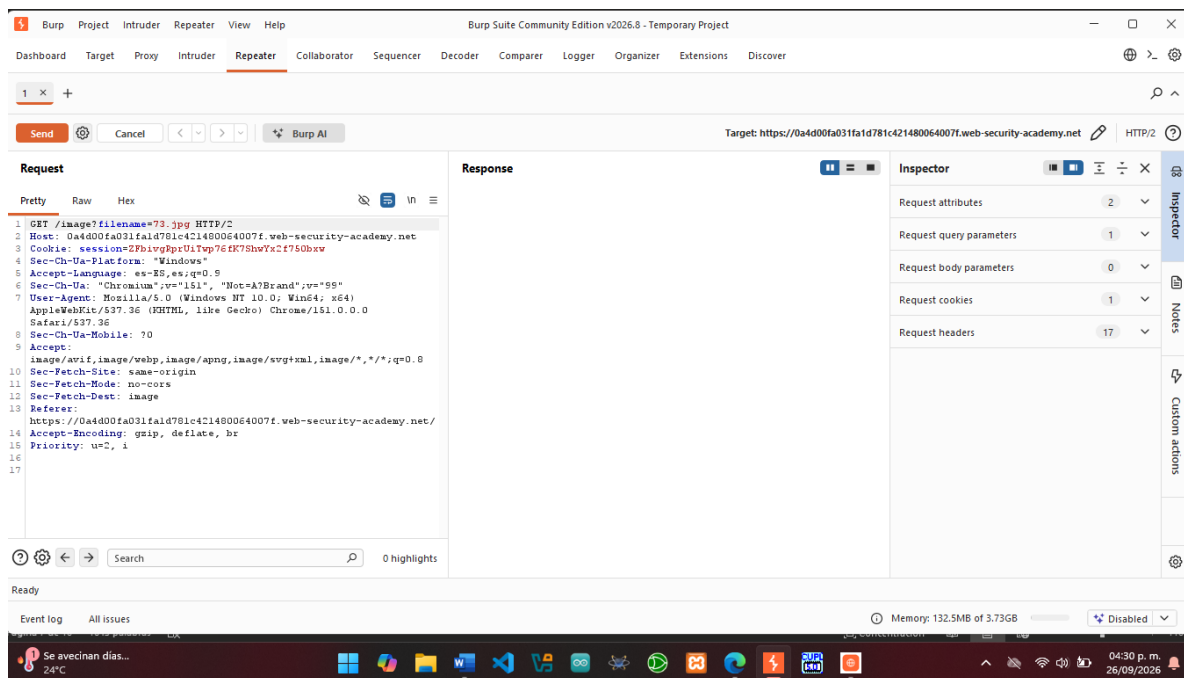
Clear history

Copy URL

Copy as curl command (bash)

Copy links in response

Save item



4. Modificación del parámetro vulnerable

Dentro de **Repeater**, se modificó el valor original del parámetro **filename**, sustituyendo el nombre de la imagen por el siguiente payload: **../%252f..%252f..%252fetc/passwd**.

Posteriormente se envió la petición modificada al servidor.

La petición modificada quedó conceptualmente de la siguiente forma: **GET /image?filename=../%252f..%252f..%252fetc/passwd**.

A diferencia de una secuencia convencional **../**, en este caso los caracteres utilizados como separadores se encuentran doblemente codificados, permitiendo que la entrada evada la validación inicial y que las secuencias de traversal aparezcan posteriormente durante el proceso de decodificación.

5. Análisis de la respuesta

El servidor respondió con: **HTTP/2 200 OK**.

En el cuerpo de la respuesta se observó información correspondiente al archivo **/etc/passwd**.

Esto confirmó que el payload doblemente codificado había sido procesado correctamente y que la aplicación permitía acceder a un archivo situado fuera del directorio destinado al almacenamiento de imágenes.

Explicación del payload

El payload utilizado para explotar la vulnerabilidad fue:

`..%252f..%252f..%252fetc/passwd.`

Su funcionamiento se basa en la **doble codificación URL** de los separadores utilizados en las secuencias de recorrido de directorios.

Normalmente, una secuencia de traversal se representa como: `../`. El carácter `/` puede codificarse mediante URL encoding como: `%2f`.

A su vez, el carácter `%` se representa como `%25`. Por esta razón, al aplicar una segunda codificación, `%2f` se transforma en: `%252f`.

Así, cada fragmento: `..%252f` representa una versión doblemente codificada de: `../`. El payload repite esta construcción tres veces: `..%252f..%252f..%252f`.

Después de las etapas de decodificación realizadas por la aplicación, estas representaciones pueden convertirse nuevamente en secuencias de recorrido, produciendo de forma efectiva: `../../..`.

Finalmente se añade: `etc/passwd`, por lo que la ruta resultante permite intentar acceder a: `../../..etc/passwd.`

Las tres secuencias permiten retroceder desde el directorio utilizado para almacenar las imágenes y alcanzar el archivo `/etc/passwd`.

Por lo tanto, el bypass funciona porque la aplicación realiza una decodificación adicional después de la validación inicial, provocando que una entrada que inicialmente no contiene los separadores en su forma convencional termine convirtiéndose en una ruta de traversal válida.

Mitigación recomendada

Para prevenir este tipo de vulnerabilidad, la aplicación debe evitar utilizar directamente entradas proporcionadas por el usuario para construir rutas dentro del sistema de archivos.

Una medida recomendable consiste en utilizar **identificadores internos o una lista de valores permitidos (allowlist)** para seleccionar los archivos disponibles. De esta forma, el usuario no proporciona directamente el nombre o la ruta que será procesada por el sistema.

Cuando sea indispensable utilizar datos proporcionados por el usuario, la aplicación debe **normalizar y decodificar completamente la entrada antes de realizar las comprobaciones de seguridad**. La validación debe aplicarse sobre la representación final que será utilizada para acceder al sistema de archivos, evitando inconsistencias entre las etapas de validación y procesamiento.

Después de normalizar y validar la entrada, la aplicación debe construir la ruta a partir de un directorio base controlado, obtener su ruta canónica y comprobar que la ruta resultante continúe perteneciendo al directorio autorizado antes de acceder al archivo.

También debe aplicarse el **principio de mínimo privilegio**, de manera que el proceso de la aplicación tenga acceso únicamente a los archivos estrictamente necesarios para su funcionamiento. Esto reduce el impacto potencial en caso de que un mecanismo de validación sea evadido.

Referencias

PortSwigger. (s. f.). *Path traversal*. Web Security Academy.

[Path Traversal — PortSwigger Web Security Academy](#)

PortSwigger. (s. f.). *Path traversal learning path*. Web Security Academy.

[Path Traversal Learning Path — PortSwigger](#)

López Contreras, S. (2026). *CNO IV (T37A): Seguridad Informática. Actividad 09: File Path Traversal – Traversal Sequences Stripped Non-Recursively*. Universidad Politécnica de San Luis Potosí.